Credit Risk Prediction Platform
Final Report

1 Introduction

This project presents an end-to-end Credit Risk Prediction Platform that leverages machine learning and MLOps tools to predict the probability of loan default. The system is designed to provide a seamless interface for non-technical users while maintaining a scalable and reproducible backend pipeline.

2 Problem Statement

Financial institutions need reliable and interpretable systems to evaluate loan applicants. Incorrect decisions can lead to financial losses. This project builds a machine learning system that predicts default probability and assists in decision-making.

3 Dataset Description

The dataset used in this project is from the Kaggle competition:

Home Credit Default Risk Dataset

3.1 Overview

The dataset contains anonymized information about loan applicants, including:

- Demographic information
- Financial history
- Credit bureau records
- Previous loan data

3.2 Key Files Used

- `application_train.csv` – main training dataset
- `bureau.csv` – credit history
- `bureau_balance.csv` – monthly credit balance

3.3 Target Variable

- `TARGET = 1` → Default
- `TARGET = 0` → Non-default

4 System Architecture

The system consists of the following components:

- Streamlit UI (Frontend)
- FastAPI (Backend)
- LightGBM Model
- DVC Pipeline (Data Versioning & Reproducibility)
- MLflow (Experiment Tracking)
- Airflow (Workflow Orchestration)
- Prometheus & Grafana (Monitoring)

4.1 Architecture Diagram

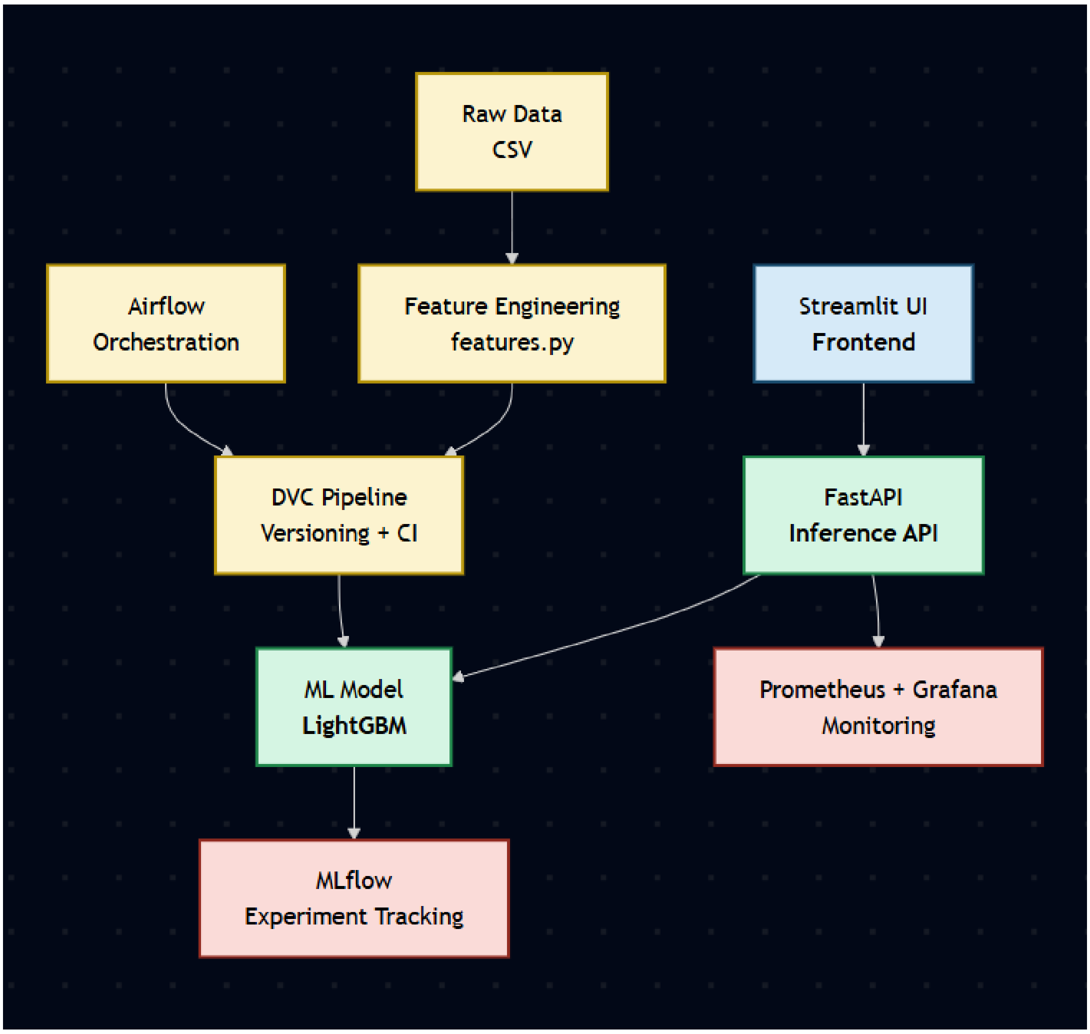

Figure 1: System Architecture

5 Data Processing & Feature Engineering

The raw data undergoes multiple preprocessing steps:

- Handling missing values
- Label encoding of categorical variables
- Feature engineering from bureau data
- Aggregation of historical credit data

Custom features such as:

- Debt-to-credit ratio
- Overdue ratios
- Active credit ratios

are created to improve model performance.

6 Machine Learning Model

A LightGBM classifier is used due to its efficiency and performance on tabular data.

6.1 Key Characteristics

- Handles large datasets efficiently
- Supports categorical features
- Works well with imbalanced data

6.2 Training Strategy

- Stratified K-Fold Cross Validation
- Early stopping to prevent overfitting
- Hyperparameter tuning

7 System Workflow

1. User inputs data via Streamlit UI
2. UI sends request to FastAPI
3. FastAPI preprocesses input and loads model
4. Model predicts default probability
5. Result is returned and displayed
6. Metrics are recorded for monitoring
7. Airflow triggers pipeline when new data arrives
8. DVC executes pipeline stages
9. MLflow logs experiments

8 MLOps Pipeline

8.1 DVC Pipeline

The pipeline consists of multiple stages:

- Feature Engineering
- Data Splitting
- Model Training
- Evaluation

DVC ensures reproducibility and version control for data and models.

8.2 MLflow

MLflow is used to:

- Track experiments
- Store model artifacts
- Compare performance across runs

8.3 Airflow

Airflow automates the pipeline:

- Detects new data
- Triggers DVC pipeline
- Runs evaluation

9 Monitoring

The system integrates Prometheus and Grafana for monitoring.

- API request counts tracked using Prometheus
- Prediction scores monitored
- System performance visualized in Grafana dashboards

Although full dashboard visualization is environment-dependent, the metrics pipeline has been successfully implemented and validated.

10 API Design

10.1 POST /predict

Input:

```
{
  "data": { feature_dict }
}
```

Output:

```
{
  "default_probability": float,
  "decision": "APPROVE" | "REJECT"
}
```

11 UI Design

The UI is built using Streamlit and provides an intuitive interface.

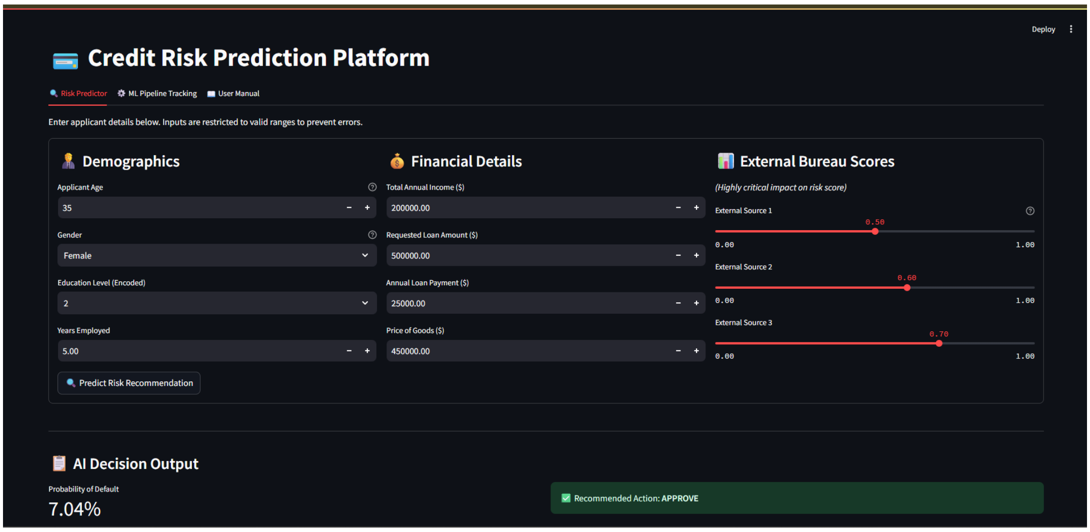

Figure 2: User Interface

12 Pipeline Visualization

Figure 3: DVC Pipeline DAG

13 Experiment Tracking

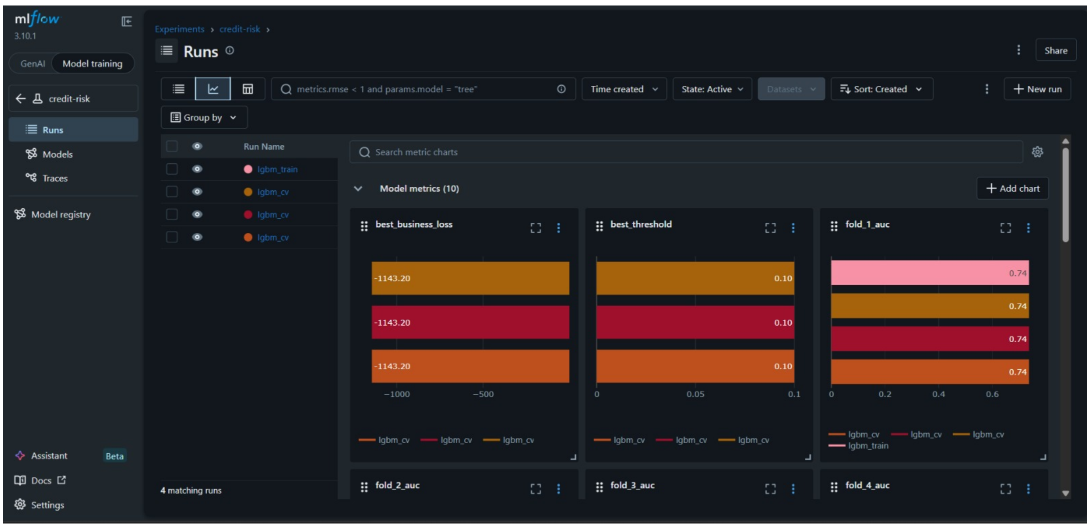

Figure 4: MLflow UI

14 Workflow Orchestration

Figure 5: Airflow DAG

15 Testing

15.1 Test Cases

- Valid input returns prediction
- Missing features handled correctly
- API failures handled gracefully

15.2 Test Results

- Total Test Cases: 5
- Passed: 5
- Failed: 0

16 Results

The model demonstrates strong performance in predicting loan defaults and provides meaningful decision support.

17 Challenges Faced

- Handling imbalanced dataset
- Integrating multiple MLOps tools
- Debugging monitoring setup

18 Future Work

- Improve monitoring dashboards
- Deploy system using Docker
- Enhance model accuracy

19 Conclusion

This project demonstrates a complete end-to-end MLOps pipeline integrating data processing, model training, deployment, and monitoring.